Handoff técnico: ROS 2 Jazzy embarcado no Android Automotive (AAOS)

28 de out. de 2026 · @Someone

O ROS 2 Jazzy (rclcpp + CycloneDDS) roda no AAOS 14 do emulador, como executável nativo e dentro de um APK, trocando mensagens com um host Linux por CycloneDDS em TCP. Para o time VIPEE-ROS2AAOSBridge, que vai reproduzir isto do zero e embutir o ROS 2 em HALs de vendor (VHAL e EVS).

Como ler este documento

Cada afirmação cita `arquivo:linha`, um comando ou um log. **NÃO VERIFICADO** = não há evidência no workspace nem nos logs; o motivo aparece ao lado.

Placeholder	Significado
`<WS>`	o workspace `ros2_android_ws`
`<ROS2_SRC>`	árvore-fonte do ROS 2 Jazzy usada no build; tem `src/` e `install/` nativos do host
`<SIRV_SRC>`	repositório SIRV-Simulator, fonte do `sirv_msgs`
`<SURROUND_SRC>`	projeto SurroundVisionROS2-main
`<ANDROID_SDK>`	Android SDK do host
`<NDK>`	`<ANDROID_SDK>/ndk/27.0.12077973`
`<HOST_IP>`	IP do host na LAN
`<DEVICE_IP>`	IP do device real na LAN
`<DEVICE_SERIAL>`	serial adb do device real

Fontes das citações.

- **O workspace não é um repositório git**, então não há hashes de commit dele. O histórico vem de três fontes:
  - `log/build_*/logger_all.log`: 58 invocações do `colcon` com os argumentos exatos, de `log/build_2026-09-13_14-19-36` a `…_15-22-23`.
  - `log/build_*/<pkg>/command.log`: o ambiente de cada build.
  - `scratch/*.log`: logs de build e do emulador.
- Os comandos `adb`, `emulator` e `cmake` e as saídas deles não ficaram em arquivo do workspace. Vieram dos transcripts das sessões do Claude Code que fizeram o trabalho, citados como `[T:<id>@<hora UTC>]`:
  - `T:14b4d291`: bring-up de 2026-09-13/14.
  - `T:6d19af38`: teste em device arm64 real, 2026-09-16.
- Os transcripts são locais e não acompanham o workspace. Peça cópia se precisar auditar.

1. Resumo

O ROS 2 Jazzy (rclcpp + CycloneDDS) foi cross-compilado com o NDK r27 para Android API 34, em arm64-v8a e x86_64. No AAOS 14 do emulador x86_64 ele roda como executável nativo via `adb shell` e embutido num APK, e nos dois modos troca mensagens com um host Linux por **CycloneDDS em TCP dial-out**. No device arm64 real (AAOS 16, segundo o transcript), o `rclcpp::init` e a descoberta DDS funcionaram.

#	O que	Evidência
E1	Talker nativo rodando no emulador AAOS 14 x86_64	`[T:14b4d291@17:52:24]`:

		`[android_bringup_talker]: publishing: 'hello from android #0'`, `#1`, …
E2	Pub/sub entre 2 processos no mesmo emulador	`[T:14b4d291@17:53:07]: [android_bringup_listener]: heard: 'hello from android #1'` … `#5`
E3	**Host → Android:** `ros2 topic pub` no host, listener nativo no emulador	`[T:14b4d291@18:42:13]: [android_bringup_listener]: heard: 'hello from host, domain fixed'`; no host, `ss -tlnp` mostrou `LISTEN 0.0.0.0:7400 users:(("ros2",…))`
E4	`ros2 bag play` no host → `compressed_frame_saver` no emulador	**1658 JPEGs** em `scratch/received_frames/frames/`: front 226, left 226, right 226, rear 251, inside 377, ground 352; `front_0000.jpg` tem 1920×1080
E5	rclcpp **dentro do processo do APK**	logcat `Ros2Bridge: rclcpp::init OK` / `node 'surround_vision_bridge' criado, entrando em spin()`; shutdown limpo `spin() retornou → rclcpp::shutdown OK → stopped` (`[T:14b4d291@00:41:52]`)
E6	Device arm64 real `<DEVICE_SERIAL>`	logcat `CYCLONEDDS_URI setado (dial-out TCP, domain=25, peers=…<HOST_IP>:7500… <HOST_IP>:7501)`, `rclcpp::init OK`, `RosCameraQuad: assinado /sirv/sensor/fisheye/front/image/compressed`; no host, `ss -tn` mostrou `ESTAB <HOST_IP>:7501 <DEVICE_IP>:36087` (`[T:6d19af38@14:26:06]`, `@14:26:37`)

Não há captura de `ros2 topic echo` do lado Android: lá foram usados nós próprios (`listener`, `compressed_frame_saver`) e logcat. A chegada de **imagens** no device arm64 real ficou **NÃO VERIFICADA**: no último teste o simulador não publicava nem no próprio host (`[T:6d19af38@14:28:13]`).

2. Ambiente

Não houve build AOSP: foi usada a imagem Google AAOS API 34-ext9 no emulador e o NDK r27 para cross-compilar.

Item	Valor	Fonte
SO do host	Ubuntu 24.04.5 LTS, kernel 7.0.0-34-generic, x86_64	`lsb_release -ds`, `uname -r`
AOSP: tag do manifest / lunch target	**Não aplicável / NÃO VERIFICADO.** Nenhuma árvore AOSP foi compilada; não existe `Android.bp`, `Android.mk`, `.rc`, `.te` nem manifest repo no workspace (`find` vazio)	—
Imagem do emulador	`system-images/android-34-ext9/android-automotive/x86_64/`, tag `android-automotive,google_apis`, AVD `aaos14`	`~/.android/avd/aaos14.avd/config.ini`; `scratch/emulator.log:3`
Versão Android reportada	`ro.build.version.release = 14`, `ro.product.cpu.abi = x86_64`	`[T:14b4d291@17:45:45]`
Emulador	Android emulator 36.6.11.0 (build_id 15507667), gfxstream e SwiftShader	`scratch/emulator.log:1-2`, `scratch/emulator5.log`
AVD arm64	**Não roda neste host:** `FATAL: Avd's CPU Architecture 'arm64' is not supported by the QEMU2 emulator on x86_64 host`	`scratch/emulator_arm64.log`
Device arm64 real	Raspberry Pi 4 com AAOS 16, segundo o transcript; fingerprint e `ro.build.*` **NÃO VERIFICADOS** (não capturados)	`[T:6d19af38]`

	VERIFICADOS (não capturados)	
Arquiteturas	`arm64-v8a` (alvo real) e `x86_64` (só para validar no emulador)	`android_arm64.toolchain.cmake:1-5`, `android_x86_64.toolchain.cmake:1-8`
NDK	**27.0.12077973** (r27)	`build_android/rclcpp/CMakeFiles/*/CMakeCXXCompiler.cmake`; `surround_vision_android/app/build.gradle.kts:9`
API mínima	`android-34`	`android_arm64.toolchain.cmake:4`
STL	`c++_shared`	`android_arm64.toolchain.cmake:5`
CMake	3.28.3 no colcon/host; 3.22.1 no Gradle	`cmake --version`; `build.gradle.kts:48`
colcon	`/usr/bin/colcon`; colcon-bash 0.5.0, colcon-argcomplete 0.3.3	`log/*/logger_all.log`; `colcon version-check`
App	compileSdk 34, minSdk 30, targetSdk 34, Kotlin 2.0.0, JVM 17	`build.gradle.kts:3,8,13-14,65-71`
OpenCV	Android SDK oficial 4.10.0 (`opencv-4.10.0-android-sdk.zip`)	colcon cv_bridge `-DOpenCV_DIR=…/OpenCV-android-sdk/sdk/native/jni`

Atenção: no shell atual, `ANDROID_NDK_HOME` aponta para `$ANDROID_HOME/ndk` (diretório com várias versões). Isso **não** funciona com o toolchain file, que faz `include($ENV{ANDROID_NDK_HOME}/build/cmake/android.toolchain.cmake)`. Todos os builds exportaram `ANDROID_NDK_HOME=<NDK>` explicitamente (`[T:14b4d291@17:35:35]` e outros).

3. ROS 2

Distro Jazzy, fontes em `<ROS2_SRC>/src`, RMW `rmw_cyclonedds_cpp` (CycloneDDS 0.10.5). O `CMAKE_HOME_DIRECTORY` de cada pacote confirma a origem, por exemplo `build_android/rclcpp/CMakeCache.txt` → `<ROS2_SRC>/src/ros2/rclcpp/rclcpp`.

Versões dos repos-fonte (`git -C <repo> describe --tags --always; rev-parse --short HEAD`):

Repo	Tag/describe	Commit	Branch
ros2/rcutils	6.7.2	2cd95ae	jazzy
ros2/rclcpp	28.1.5-2-g99b3803c	99b3803c	jazzy
ros2/rcl	9.2.4	7927b76	jazzy
ros2/rmw	7.3.1	c90c297	jazzy
ros2/rmw_cyclonedds	2.2.2	7645474	jazzy
eclipse-cyclonedds/cyclonedds	0.10.5-1-g454df330	454df330	releases/0.10.x
eclipse-iceoryx/iceoryx	v2.0.6-9-gebdf38728	ebdf38728	release_2.0 (só headers/cmake; SHM desligado)
ros2/rcpputils	2.11.0	747b92f	jazzy
ros2/rosidl	4.6.4-1-gb56ff98	b56ff98	jazzy
ros2/rosidl_typesupport	3.2.2	dc851e0	jazzy
ros2/rosidl_typesupport_fastrtps	3.6.0	8a25101	jazzy
eProsima/Fast-CDR	v2.0.0-45-g5cc8c55	5cc8c55	2.2.x
eProsima/foonathan_memory_vendor	v1.3.1-6-g716cfd8	716cfd8	master
ros2/rcl_logging	3.1.0-1-gda51595	da51595	jazzy
ros2/common_interfaces	5.3.5-1-g784c489	784c489	jazzy
ros2/libyaml_vendor	1.6.3 (**modificado**, ver §4)	dbba913	jazzy

ros2/yaml_cpp_vendor	9.0.1	867dd6b	jazzy
ros-perception/vision_opencv (cv_bridge)	4.1.0-13-g50f6f1f	50f6f1f	**rolling**
tinyxml2_vendor / spdlog_vendor / console_bridge_vendor	0.9.1 / 1.6.1 / 1.7.1	85acf25 / dfd75d8 / abb96a9	jazzy; sem modificação, spdlog nem é compilado

Para recriar a árvore: **NÃO VERIFICADO.** Não há `ros2.repos` salvo em `<ROS2_SRC>` (`ls *.repos` vazio). Use o `ros2.repos` oficial do Jazzy e faça checkout dos commits acima.

Pacotes cross-compilados. São 96 entradas em `install_android/` (inclui pacotes só CMake e de ferramenta). Os que geram `.so` para o device:

```
rcutils rcpputils rmw rmw_implementation rmw_dds_common rmw_cyclonedds_cpp cyclonedds
rcl rcl_yaml_param_parser rcl_logging_interface rcl_logging_noop rclcpp
libstatistics_collector tracetools ament_index_cpp libyaml_vendor yaml_cpp_vendor
fastcdr foonathan_memory_vendor rosidl_runtime_c rosidl_runtime_cpp
rosidl_typesupport_c rosidl_typesupport_cpp rosidl_typesupport_introspection_c
rosidl_typesupport_introspection_cpp rosidl_typesupport_fastrtps_c
rosidl_typesupport_fastrtps_cpp rosidl_dynamic_typesupport
builtin_interfaces std_msgs geometry_msgs sensor_msgs diagnostic_msgs action_msgs
service_msgs statistics_msgs rosgraph_msgs rcl_interfaces type_description_interfaces
unique_identifier_msgs sirv_msgs cv_bridge surround_vision
```

No total são 124 `.so`, 184 MB (`find install_android -name '*.so' | wc -l` e `du -cb`).

RMW.

- O build do núcleo **pula** `fastrtps rmw_fastrtps_cpp rmw_fastrtps_dynamic_cpp rmw_fastrtps_shared_cpp rmw_connextdds …` (`--packages-skip` em `log/build_2026-09-13_14-38-05/logger_all.log`).
- Os consumidores passam `-DRMW_IMPLEMENTATION=rmw_cyclonedds_cpp` (`bringup_talker/build_x86_64/CMakeCache.txt`), e em runtime exporta-se `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp` (`[T:14b4d291@17:52:24]`).
- O `fastcdr` e o `rosidl_typesupport_fastrtps_*` **são** compilados, porque o rosidl gera esse typesupport; o Fast DDS em si não.

Por que CycloneDDS: a justificativa escrita é **NÃO VERIFICADA**. O registrado: o CycloneDDS permite forçar TCP com peers unicast via `CYCLONEDDS_URI`, inclusive XML inline (`surround_vision_android/app/cpp/ros2/Ros2Bridge.cpp:35-63`), e isso resolveu o NAT do emulador (§7). Zenoh e Fast DDS não foram testados no Android.

Logging: `-DRCL_LOGGING_IMPLEMENTATION=rcl_logging_noop`; `spdlog_vendor` e `rcl_logging_spdlog` são pulados. Consequência: o `RCLCPP_INFO` vai para stdout/stderr, não para o logcat; o app usa `__android_log_print` direto (`Ros2Bridge.cpp:7-9`).

4. Sistema de build

O ROS 2 foi compilado com colcon cruzado e o toolchain CMake do NDK, usando `c++_shared` do NDK; Soong não foi usado e há um único patch (libyaml_vendor).

4.1 Tipo de build

- **Soong (Android.bp): não usado.** Nenhum `Android.bp` / `Android.mk` no workspace.
- **colcon cruzado + toolchain CMake do NDK** para o ROS 2: `--cmake-args --toolchain <WS>/android_arm64.toolchain.cmake`.
- **CMake puro** (`cmake -S/-B --toolchain`) para os nós de teste em `bringup_talker/` (`[T:14b4d291@17:40:59]`).
- **Gradle + externalNativeBuild CMake** para o APK (`surround_vision_android/app/build.gradle.kts:24-50`).

Toolchain file arm64, `android_arm64.toolchain.cmake:1-20`:

```
set(ANDROID_ABI arm64-v8a CACHE STRING "" FORCE)
set(ANDROID_PLATFORM android-34 CACHE STRING "" FORCE)
set(ANDROID_STL c++_shared CACHE STRING "" FORCE)
```

```
set(ANDROID_STL c++_shared CACHE STRING "" FORCE)
if(NOT DEFINED ENV{ANDROID_NDK_HOME})
  message(FATAL_ERROR "ANDROID_NDK_HOME is not set")
endif()
include("$ENV{ANDROID_NDK_HOME}/build/cmake/android.toolchain.cmake")
# O toolchain do NDK restringe find_* ao sysroot (ONLY); as ferramentas ament_cmake
# do host ficam fora do sysroot -> BOTH.
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM BOTH)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY BOTH)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE BOTH)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE BOTH)
```

`android_x86_64.toolchain.cmake` é idêntico, com `ANDROID_ABI x86_64` (`set` nas linhas 6-8, `include` na 14, `BOTH` nas 16-19).

4.2 Sequência de build executada

Ordem real, extraída de `log/build_*/logger_all.log`.

Ambiente do núcleo (`[T:14b4d291@17:35:35]`). As ferramentas ament/rosidl vêm do install nativo do host numa lista curada, **sem** `source setup.bash`:

```
export ANDROID_NDK_HOME=<NDK>
HOST_INSTALL=<ROS2_SRC>/install
PKGS="ament_package ament_cmake_core ament_cmake ament_cmake_export_definitions \
ament_cmake_export_include_directories ament_cmake_export_libraries
ament_cmake_export_link_flags \
ament_cmake_include_directories ament_cmake_libraries ament_cmake_python ament_cmake_version \
ament_cmake_export_dependencies ament_cmake_export_interfaces ament_cmake_export_targets \
ament_cmake_target_dependencies ament_cmake_test ament_cmake_vendor_package
ament_cmake_gen_version_h \
ament_cmake_ros ros_environment rosidl_adapter rosidl_parser rosidl_pycommon rosidl_cli \
python_cmake_module ament_index_python rpyutils ament_cmake_gmock ament_cmake_gtest
ament_cmake_pytest"
P=""; for pkg in $PKGS; do P="$P:$HOST_INSTALL/$pkg"; done
echo "${P#:}" > <WS>/scratch/host_prefix_path.txt          # [T:14b4d291@17:36:05]
export CMAKE_PREFIX_PATH="$(cat <WS>/scratch/host_prefix_path.txt)"
export AMENT_PREFIX_PATH="$CMAKE_PREFIX_PATH"
cd <WS>
```

Por que a lista curada:

- O `source <ROS2_SRC>/install/setup.bash` expõe cerca de 400 pacotes x86_64/glibc, e o rosidl passa a descobrir `rosidl_generator_py` e Fast-RTPS do host como extensões (`[T:14b4d291@17:31:21]`).
- O `command.log` do rclcpp confirma `CMAKE_PREFIX_PATH` / `AMENT_PREFIX_PATH` = pacotes de `install_android/*` seguidos dos do host (`log/build_2026-09-13_14-38-05/rclcpp/command.log`).

Lista de skip e flags do núcleo, cópia literal de `log/build_2026-09-13_14-38-05/logger_all.log`:

```
SKIP="ament_lint ament_pycodestyle ament_flake8 ament_mypy ament_pep257 ament_cppcheck
ament_cpplint \
ament_copyright ament_xmllint ament_uncrustify uncrustify_vendor ament_cmake_uncrustify
ament_cmake_cppcheck \
ament_cmake_cpplint ament_cmake_flake8 ament_cmake_pep257 ament_cmake_xmllint
ament_cmake_copyright \
ament_cmake_lint_cmake ament_lint_cmake ament_cmake_google_benchmark gtest_vendor gmock_vendor
\
google_benchmark_vendor osrf_testing_tools_cpp performance_test_fixture domain_coordinator
launch launch_xml \
launch_yaml launch_testing launch_testing_ament_cmake rosidl_generator_py rpyutils
rosidl_generator_rs fastrtps \
rmw_connextdds rmw_connextddsmicro rti_connext_dds_cmake_module rmw_fastrtps_cpp
rmw_fastrtps_dynamic_cpp \
rmw_fastrtps_shared_cpp rosidl_dynamic_typesupport_fastrtps rmw_connextdds_common spdlog_vendor
\
rcl_logging_spdlog test_msgs test_interface_files iceoryx_hoofs iceoryx_posh iceoryx_binding_c
```

```
osrf_pycommon \
ament_lint_common mimick_vendor ament_index_python"
CORE_ARGS="-DBUILD_TESTING=OFF -DENABLE_SSL=OFF -DENABLE_SECURITY=OFF -DENABLE_SHM=OFF -
DBUILD_IDLC=OFF \
-DBUILD_DDSPERF=OFF -DBUILD_EXAMPLES=OFF -DBUILD_DOCS=OFF -
DRCL_LOGGING_IMPLEMENTATION=rcl_logging_noop"
```

arm64-v8a, com `--build-base build_android --install-base install_android` e `TC=--toolchain <WS>/android_arm64.toolchain.cmake`:

```
# 1) prova da cadeia                          (log/build_2026-09-13_14-19-36 … 14-21-07)
colcon build --base-paths <ROS2_SRC>/src --build-base build_android --install-base
install_android \
  --packages-select rcutils --cmake-args $TC -DBUILD_TESTING=OFF --event-handlers
console_direct+

# 2) libyaml_vendor (precisa do patch P1)    (log/build_2026-09-13_14-23-39 … 14-28-52)
colcon build --base-paths <ROS2_SRC>/src --build-base build_android --install-base
install_android \
  --packages-select libyaml_vendor --cmake-args $TC -DBUILD_TESTING=OFF \
  -DCMAKE_DISABLE_FIND_PACKAGE_yaml=ON --event-handlers console_direct+

# 3) núcleo até rclcpp (76 pacotes)          (log/build_2026-09-13_14-38-05)
colcon build --base-paths <ROS2_SRC>/src --build-base build_android --install-base
install_android \
  --packages-up-to rclcpp --packages-skip $SKIP --cmake-args $TC $CORE_ARGS --event-handlers
console_direct+

# 4) mensagens                                (log/…_14-45-20, …_14-55-15)
colcon build … --packages-up-to geometry_msgs --packages-skip $SKIP --cmake-args $TC $CORE_ARGS
…
colcon build … --packages-up-to sensor_msgs   --packages-skip $SKIP --cmake-args $TC $CORE_ARGS
…

# 5) yaml-cpp real (sem achar o do sistema)  (log/…_14-56-04)
colcon build … --packages-select yaml_cpp_vendor --cmake-args $TC -DBUILD_TESTING=OFF \
  -DCMAKE_DISABLE_FIND_PACKAGE_yaml-cpp=ON --event-handlers console_direct+

# 6) mensagem própria do projeto + dependências  (log/…_14-56-39 … 14-58-53)
colcon build --base-paths <ROS2_SRC>/src <SIRV_SRC>/sirv_ros_pkgs --build-base build_android \
  --install-base install_android --packages-up-to diagnostic_msgs sirv_msgs --packages-skip
$SKIP \
  --cmake-args $TC $CORE_ARGS --event-handlers console_direct+

# 7) cv_bridge com OpenCV Android + headers boost    (log/…_15-01-30)
colcon build --base-paths <ROS2_SRC>/src --build-base build_android --install-base
install_android \
  --packages-select cv_bridge --cmake-args $TC -DBUILD_TESTING=OFF \
  -DOpenCV_DIR=<WS>/scratch/opencv_android/OpenCV-android-sdk/sdk/native/jni \
  "-DCMAKE_CXX_FLAGS=-isystem <WS>/scratch/boost_headers_only" --event-handlers console_direct+

# 8) nó do projeto (fast_image_republisher)  (log/…_15-22-23)
colcon build --base-paths "<SURROUND_SRC>" --build-base build_android --install-base
install_android \
  --packages-select surround_vision --cmake-args $TC -DBUILD_TESTING=OFF \
  -DOpenCV_DIR=<WS>/scratch/opencv_android/OpenCV-android-sdk/sdk/native/jni \
  -Dyaml-cpp_DIR=<WS>/install_android/yaml_cpp_vendor/opt/yaml_cpp_vendor/lib/cmake/yaml-cpp
```

x86_64: mesma sequência com `--build-base build_android_x86_64 --install-base install_android_x86_64` e `TC=--toolchain <WS>/android_x86_64.toolchain.cmake`:

- `log/…_14-47-33` até `…_14-50-42`: `--packages-up-to rclcpp geometry_msgs`.
- `…_14-48-56`: libyaml_vendor.
- `…_15-06-24`: `--packages-up-to sensor_msgs sirv_msgs yaml_cpp_vendor` com `-DCMAKE_DISABLE_FIND_PACKAGE_yaml-cpp=ON`.
- `…_15-07-25`: cv_bridge.
- `…_15-07-38` até `…_15-22-16`: surround_vision.

Nós de teste (`bringup_talker/` , CMake puro; `[T:14b4d291@17:40:59]` , `@17:52:59`). O build precisa de um `AMENT_PREFIX_PATH` apontando **só** para o install Android; sem ele, o `get_available_rmw_implementations` enxerga o `rmw_fastrtps_cpp` do host (`[T:14b4d291@17:40:11]`).

```
cd <WS>/install_android_x86_64
P=""; for d in */; do pkg="${d%/}"; [ "$pkg" = COLCON_IGNORE ] && continue
  if [ -f "$pkg/share/$pkg/package.xml" ] || [ -f "$pkg/share/ament_index/resource_index/
packages/$pkg" ]; then
    P="$P:<WS>/install_android_x86_64/$pkg"; fi; done
echo "${P#:}" > <WS>/scratch/android_x86_64_ament_prefix_path.txt      # [T:14b4d291@17:51:28]

export ANDROID_NDK_HOME=<NDK>
export AMENT_PREFIX_PATH="$(cat <WS>/scratch/android_x86_64_ament_prefix_path.txt)"
export CMAKE_PREFIX_PATH="<WS>/install_android_x86_64"
cmake -S <WS>/bringup_talker -B <WS>/bringup_talker/build_x86_64 \
  --toolchain <WS>/android_x86_64.toolchain.cmake \
  -DCMAKE_PREFIX_PATH=<WS>/install_android_x86_64 -DRMW_IMPLEMENTATION=rmw_cyclonedds_cpp
cmake --build <WS>/bringup_talker/build_x86_64 -- -j$(nproc)
```

Saída esperada do configure (`[T:14b4d291@17:51:28]` , `@17:52:59`):

```
-- Using all available rosidl_typesupport_c:
rosidl_typesupport_fastrtps_c;rosidl_typesupport_introspection_c
-- Using all available rosidl_typesupport_cpp:
rosidl_typesupport_fastrtps_cpp;rosidl_typesupport_introspection_cpp
-- Using RMW implementation 'rmw_cyclonedds_cpp'
```

APK (`surround_vision_android/`), comando `./gradlew :app:assembleDebug` . O `app/CMakeLists.txt` :

- faz `unset(ENV{CMAKE_PREFIX_PATH})` , `AMENT_PREFIX_PATH` , `ROS_PACKAGE_PATH` e `COLCON_PREFIX_PATH` (linhas 12-15), contra a poluição do shell com ROS do host;
- monta o `CMAKE_PREFIX_PATH` varrendo `install_android[_x86_64]/*` e `*/opt/*` (linhas 73-100);
- recoloca `ENV{AMENT_PREFIX_PATH}` com a mesma lista (linhas 107-108), necessário para o `get_used_typesupports.cmake` ;
- faz `find_package(rclcpp|sensor_msgs|sirv_msgs|yaml-cpp CONFIG REQUIRED)` (linhas 117-120) e o link (linhas 122-142).

4.3 libc++ e onde os binários linkam

- **STL: `c++_shared` do NDK r27**, em `android_arm64.toolchain.cmake:5` , `android_x86_64.toolchain.cmake:8` e `build.gradle.kts:26` .
- `llvm-readelf -d` em `install_android/rclcpp/lib/librclcpp.so` (aarch64) e em `push_x86_64/talker` (intérprete `/system/bin/linker64`) mostra NEEDED `libc++_shared.so` , `libc.so` , `libm.so` , `libdl.so` e, fora isso, só libs ROS.
- O `libc++_shared.so` vem do NDK: `cp <NDK>/toolchains/llvm/prebuilt/linux-x86_64/sysroot/usr/lib/x86_64-linux-android/libc++_shared.so <WS>/push_x86_64/` (`[T:14b4d291@17:52:04]`).
- **A libc++ da plataforma não é usada, e nada linka em módulo de plataforma (vendor/system).** Tudo roda como executável em `/data/local/tmp/ros2_android/` com `LD_LIBRARY_PATH` , ou como `.so` dentro do APK (`jniLibs`).
- Comportamento dentro de processo `vendor` / `system` : **NÃO VERIFICADO** (ver §11).

4.4 Patches

#	Onde	Motivo	Diff/Origem
P1	`<ROS2_SRC>/src/ros2/libyaml_vendor` (`CMakeLists.txt` + `patches/android_cmake_min_version.patch`)	O `flags.cmake` do NDK r26+ usa `if(... IN_LIST ...)` (política CMP0057); o libyaml 0.2.5 declara `cmake_minimum_required(VERSION 3.0)` , que reseta a política e quebra a detecção do	diffs abaixo

		compilador; definir a política no toolchain não adianta (`android_arm64.toolchain.cmake:22-30`)	
P2	`<SURROUND_SRC>/CMakeLists.txt:99-109`	`-march=native` não existe no cross-compiler NDK/AArch64; desligado com `if(ANDROID)`	arquivo-fonte, sem git
P3	`<SURROUND_SRC>` (`fast_image_republisher`)	Em executável solto via `adb shell`, o `cv::cvtColor(BGR2RGB)` trava; trocado por troca manual de canais com `#if defined(ANDROID)` (`[T:14b4d291@18:23:16]`)	diff **NÃO VERIFICADO** (projeto sem git)
P4	`<SIRV_SRC>/sirv_ros_pkgs/sirv_msgs/package.xml`	Faltava `depend` de `diagnostic_msgs` (`[T:14b4d291@18:23:16]`); hoje está nas linhas 22 e 26, e em `CMakeLists.txt:29,88`; não dá para confirmar se é local ou upstream	`git status`: `M package.xml` (versão 0.0.1→3.0.0) e `M CMakeLists.txt` (+`SirvMapInfo.msg`, +`GetVersion.srv`)
—	rcutils, rclcpp, rcl, spdlog, tinyxml2, console_bridge, cyclonedds, rmw_cyclonedds	**Sem patch**: `git status --short` limpo em todos os repos de `<ROS2_SRC>/src`, exceto libyaml_vendor	—
—	handler de sinal do rclcpp	Não é patch: o app usa `initOptions.shutdown_on_signal = false` (`Ros2Bridge.cpp:69-74`), para o rclcpp não instalar handlers de SIGINT/SIGTERM; os executáveis de teste usam `rclcpp::init(argc, argv)` padrão (`bringup_talker/talker.cpp:36`)	—

Diff do P1 (`git -C <ROS2_SRC>/src/ros2/libyaml_vendor diff`):

```
--- a/CMakeLists.txt
+++ b/CMakeLists.txt
@@ -17,6 +17,8 @@ ament_vendor(libyaml_vendor
   SATISFIED ${yaml_FOUND}
   VCS_URL https://github.com/yaml/libyaml.git
   VCS_VERSION 0.2.5
+  PATCHES
+    "${CMAKE_CURRENT_SOURCE_DIR}/patches/android_cmake_min_version.patch"
   CMAKE_ARGS
     "-DCMAKE_C_FLAGS:STRING=${CMAKE_C_FLAGS}$<$<BOOL:${MSVC}>: /wd4244 /wd4267 /wd4996>"
 )
```

Arquivo novo `patches/android_cmake_min_version.patch`:

```
--- a/CMakeLists.txt
+++ b/CMakeLists.txt
@@ -1,5 +1,8 @@

-cmake_minimum_required(VERSION 3.0)
+# Bumped from 3.0 for Android NDK cross-compiles: the NDK (r26+) toolchain's
+# flags.cmake uses if(... IN_LIST ...), which needs policy CMP0057 (NEW since
+# CMake 3.3); a project-requested version below that resets it to OLD.
+cmake_minimum_required(VERSION 3.5)
 project (yaml C)
```

Flags que substituem patches (`log/build_2026-09-13_14-38-05/logger_all.log`):

- CycloneDDS sem OpenSSL, segurança, SHM/iceoryx e `idlc`: `-DENABLE_SSL=OFF -DENABLE_SECURITY=OFF -DENABLE_SHM=OFF -DBUILD_IDLC=OFF`; confirmado em `build_android/cyclonedds/CMakeCache.txt` (`ENABLE_SECURITY:BOOL=OFF`, `ENABLE_SHM:STRING=OFF`).
- `-DCMAKE_DISABLE_FIND_PACKAGE_yaml=ON` / `…yaml-cpp=ON` força os vendors a compilarem a lib própria em vez de acharem a do host.

5. Mensagens e type support

O rosidl roda no host e gera código C/C++ compilado pelo NDK; as typesupports são

carregadas por `dlopen()`, então todas as `.so` precisam ir para o device.

Como é gerado. Os geradores são Python, vindos dos pacotes `rosidl_*` do host em `host_prefix_path.txt`. Para cada pacote de mensagens são geradas e instaladas:

- `lib<pkg>__rosidl_generator_c.so`;
- `lib<pkg>__rosidl_typesupport_c.so` e `…_cpp.so`;
- `lib<pkg>__rosidl_typesupport_introspection_c.so` e `…_cpp.so`;
- `lib<pkg>__rosidl_typesupport_fastrtps_c.so` e `…_cpp.so`.

A lista vem de `ls push_x86_64` (por exemplo `libsirv_msgs__rosidl_typesupport_fastrtps_cpp.so`) e da saída do configure em §4.2. O Python (`rosidl_generator_py`) e o Rust ficam fora (`$SKIP`).

Armadilha de runtime (dlopen). Copiar só a closure do `readelf NEEDED` **não basta**. O primeiro run falhou com (`[T:14b4d291@17:52:08]`):

```
Could not load library librmw_dds_common__rosidl_typesupport_introspection_cpp.so: dlopen
error:
dlopen failed: library "librmw_dds_common__rosidl_typesupport_introspection_cpp.so" not found
```

Correção: enviar **todas** as `.so` do install (`[T:14b4d291@17:52:18]`). O APK faz o mesmo: as tasks `stageRos2LibsArm64` / `X86_64` copiam `**/*.so` (`build.gradle.kts:81-99`).

Receita para um pacote próprio (ex.: `vipee_vehicle_msgs`). Segue o padrão usado com `sirv_msgs` (§4.2 passo 6); a execução com esse pacote específico é **NÃO VERIFICADA.**

1. Pacote ament_cmake comum, com `rosidl_generate_interfaces(${PROJECT_NAME} … DEPENDENCIES std_msgs geometry_msgs …)`. O `package.xml` deve declarar **todas** as dependências de mensagem: a falta de `diagnostic_msgs` no `sirv_msgs` quebrou o build (P4).
2. Compilar com colcon, adicionando o diretório do pacote ao `--base-paths`:

   ```
   colcon build --base-paths <ROS2_SRC>/src <VIPEE_SRC> --build-base
   build_android --install-base install_android \
     --packages-up-to vipee_vehicle_msgs --packages-skip $SKIP --cmake-args
   $TC $CORE_ARGS --event-handlers console_direct+
   ```

3. Linkar o consumidor. O `sirv_msgs` **não** exporta o alvo `<pkg>::<pkg>_library`; o link é direto em `sirv_msgs::sirv_msgs__rosidl_typesupport_cpp` (`surround_vision_android/app/CMakeLists.txt:135-140`). Verifique qual alvo o seu pacote exporta.
4. Enviar todas as `.so` novas, incluindo introspection/fastrtps, mesmo sem NEEDED explícito.
5. Recompilar sempre que a definição mudar dos dois lados: o `sirv_msgs` do build tem 21 `.msg` (`install_android/sirv_msgs/share/sirv_msgs/msg`) e a fonte atual tem 23.

6. Execução no Android

O código rodou só como executável do shell do adb e dentro de um APK; nunca como processo `system` / `vendor`, serviço `init` ou HAL, e nenhuma regra SELinux foi escrita.

6.1 Onde o código roda

Modo	Processo/UID	Como	Evidência
A. Executável nativo	shell do `adb` (uid `shell`); domínio SELinux **NÃO VERIFICADO** (nunca rodado `id -Z` / `ps -Z`)	binários em `/data/local/tmp/ros2_android/`, libs em `/data/local/tmp/ros2_android/lib/`, `LD_LIBRARY_PATH`	`[T:14b4d291@17:52:24]`
B. Dentro de APK	processo do app `com.surroundvision.app` (UID de app; `untrusted_app` presumido, **NÃO VERIFICADO**)	`libsurroundvision_native.so` + libs ROS em `jniLibs`; `NativeBridge.rosStart()` em `MainActivity.onCreate`	§1 E5/E6

		(`MainActivity.kt:42`), `rosStop()` em `onDestroy` (`MainActivity.kt:142`)	
Processo `system` / `vendor` , serviço `init` (`.rc`), HAL	**Não feito.** Nenhum `.rc` , `.te` , VINTF ou Android.bp no workspace	—	

No modo B, o `Ros2Bridge` é um singleton com thread própria: `rclcpp::init` , `Node("surround_vision_bridge")` , `SingleThreadedExecutor::spin()` e cancelamento via `executor->cancel()` + `rclcpp::shutdown()` (`Ros2Bridge.cpp:18-124`). Os consumidores (`RosCameraQuad` , `RosTelemetryBridge`) usam `Ros2Bridge::waitForNode()` (`Ros2Bridge.h:31-36`) e assinam com `rclcpp::QoS(1).best_effort()` (`RosTelemetryBridge.cpp:72-81`).

6.2 SELinux e permissões

- **Nenhuma regra SELinux foi adicionada**, e o `getenforce` nunca foi consultado: **NÃO VERIFICADO.**
- Nos transcripts não aparece `avc: denied` , mas ninguém procurou ativamente; isso **não** prova ausência de negações.
- O tráfego foi TCP unicast nos dois modos (mais UDP local no modo A sem XML). Multicast no device não foi testado isoladamente.
- O APK declara `android.permission.INTERNET` (`AndroidManifest.xml:30`), a permissão relevante para sockets no modo B. As `android.car.permission.*` (`AndroidManifest.xml:12-16`) são do VHAL do app, não do DDS.
- No device real: `adb -s <DEVICE_SERIAL> install -r -g …/app-debug.apk` → `Success` ; `CAR_TURN_SIGNAL` deu "Unknown permission" no AAOS 16 (`[T:6d19af38@13:36:12]` , `@13:36:49`).

6.3 Como os binários entram

Sem PRODUCT_PACKAGES nem imagem customizada.

Modo A, por `adb push` (`[T:14b4d291@17:52:04]`):

```
adb shell mkdir -p /data/local/tmp/ros2_android/lib
adb push <WS>/push_x86_64/*.so /data/local/tmp/ros2_android/lib/
adb push <WS>/push_x86_64/talker /data/local/tmp/ros2_android/talker
adb shell chmod +x /data/local/tmp/ros2_android/talker
```

O `push_x86_64/` tem 126 arquivos, 192 MB (`ls | wc -l` , `du -sh`).

Modo B: APK com as libs achatadas pela task Gradle (`build.gradle.kts:76-99`) e `useLegacyPackaging = true` (`build.gradle.kts:52-56`). O APK debug atual tem 129 MB.

7. Rede e descoberta DDS

A descoberta funcionou com CycloneDDS só em TCP, sem multicast: o host escuta numa porta fixa e o Android disca até ele, sem `adb reverse` / `forward` .

7.1 Topologias testadas

#	Publicador	Assinante	Rede	Resultado
T1	talker no emulador	listener no mesmo emulador	loopback do guest, Cyclone default (UDP/multicast), sem XML	OK (E2)
T2	`ros2 topic pub` no host Linux nativo, Ubuntu 24.04 (**não WSL**)	listener nativo no emulador	SLIRP do emulador; device disca TCP para `10.0.2.2:7400`	OK (E3)
T3	`ros2 bag play` no host	`compressed_frame_saver` no	igual a T2	OK, 1658 frames (E4)

		emulador		
T4	UE5 + `sirv_cat793fsim` no host (domain 25)	app no emulador	TCP `10.0.2.2:7500` e `:7501`	OK, frames 480×480 compostos, ~5.7–6.1 Hz (`[T:14b4d291@02:21:58]`)
T5	simulador no host	app no **RPi4 arm64** via Wi-Fi	TCP `<HOST_IP>:7500` / `:7501`	descoberta e TCP `ESTAB` OK; imagens **NÃO VERIFICADAS** (E6)

WSL: nunca testado, **NÃO VERIFICADO**. Em modo NAT deve valer o mesmo raciocínio do SLIRP: o device precisa discar para um IP:porta alcançável do Windows, encaminhado ao WSL. É hipótese, não teste.

7.2 Como a descoberta funcionou

- **Multicast não atravessa o NAT do SLIRP** do AVD. Solução: CycloneDDS em **TCP puro, sem multicast, com peers unicast**; o host **escuta** numa porta TCP fixa e o device **disca** (`<TCP><Port>-1</Port>` , sem porta de escuta).
- Não foram usados `adb reverse` , `adb forward` , TAP nem discovery server: o `10.0.2.2` é o alias fixo do host no AVD (`Ros2Bridge.cpp:35-44` , `[T:14b4d291@18:57:11]`).
- **Device real via Wi-Fi.** Um socket Python escutando `239.255.0.1:13650` (SPDP do domain 25) só recebia multicast do próprio host: o AP não encaminhava multicast entre estações, embora unicast/ping funcionassem (`[T:6d19af38@13:48:16]`). Solução temporária: `<Peer Address="<DEVICE_IP>"/>` no host. Definitiva: o app disca TCP para `cyclonedds_host_ip` (`cat793f.yaml:27-32` , `Ros2Bridge.cpp:44`).
- **Variáveis de ambiente:** `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp` , `CYCLONEDDS_URI` e `ROS_DOMAIN_ID` . O `ROS_LOCALHOST_ONLY` não é usado (`grep -r ROS_LOCALHOST_ONLY` vazio). `ROS_DOMAIN_ID` é `0` com o sample_bag/bring-up e `25` com o simulador (`cat793f.yaml:21` , `launch_ue5.sh` , `launch_sirv_cat793fsim.sh`); no app vem do preset via `setenv` (`Ros2Bridge.cpp:31-33`).

Porta	Quem escuta	Arquivo
7400	host (bag/pub), domain 0	`cyclonedds_host.xml:11-13`
7500	UE5, domain 25	`cyclonedds_cat793f_host.xml:31-33`
7501	`sirv_cat793fsim` (disca 7500)	`cyclonedds_cat793f_sirvsim.xml:31-39`
7501/7502	testes com dois processos no device (republisher/ framesaver)	`cyclonedds_device_republisher.xml:14-16` , `cyclonedds_device_framesaver.xml:14-16`
`-1`	device só disca	`cyclonedds_device.xml:14-16` ; XML inline do app `Ros2Bridge.cpp:61`

7.3 Arquivos de configuração

`cyclonedds_host.xml` (host, escuta):

```
<CycloneDDS xmlns="https://cdds.io/config">
  <Domain id="0">
    <General><Transport>tcp</Transport><AllowMulticast>false</AllowMulticast></General>
    <Discovery><ParticipantIndex>none</ParticipantIndex></Discovery>
    <TCP><Port>7400</Port></TCP>
  </Domain>
</CycloneDDS>
```

`cyclonedds_device.xml` (device/emulador, disca; vai para `/data/local/tmp/ros2_android/`). O `<Tracing>` (linhas 17-20) é opcional, só para debug:

```
<CycloneDDS xmlns="https://cdds.io/config">
  <Domain id="0">
    <General><Transport>tcp</Transport><AllowMulticast>false</AllowMulticast></General>
```

```
    <Discovery>
      <Peers><Peer Address="10.0.2.2:7400"/></Peers>
      <ParticipantIndex>none</ParticipantIndex>
    </Discovery>
    <TCP><Port>-1</Port></TCP>
    <Tracing><Verbosity>finest</Verbosity>
      <OutputFile>/data/local/tmp/ros2_android/trace_saver.log</OutputFile></Tracing>
  </Domain>
</CycloneDDS>
```

XML **inline** gerado pelo app (`Ros2Bridge.cpp:52-63`), passado em `setenv("CYCLONEDDS_URI", uri)` antes do `rclcpp::init` , sem arquivo em disco:

```
<CycloneDDS><Domain id="<ros_domain_id>">
  <General><Transport>tcp</Transport><AllowMulticast>false</AllowMulticast></General>
  <Discovery><Peers><Peer Address="<host>:<cyclonedds_tcp_port>"/><!-- + um por
cyclonedds_tcp_extra_ports --></Peers>
  <ParticipantIndex>none</ParticipantIndex></Discovery>
  <TCP><Port>-1</Port></TCP>
</Domain></CycloneDDS>
```

O `<host>` vale `10.0.2.2` se `isEmulator()` , ou `cyclonedds_host_ip` do preset; vazio = o app não seta o URI e cai no multicast default (`Ros2Bridge.cpp:44-45`). O `isEmulator()` olha o `FINGERPRINT` (`generic` / `emulator`) ou a ABI x86/x86_64 (`MainActivity.kt:151-155`).

Também no workspace (domain 25), cada um com o modo de uso no cabeçalho:

- `cyclonedds_cat793f_host.xml` : UE5, escuta 7500.
- `cyclonedds_cat793f_sirvsim.xml` : escuta 7501 e disca `localhost:7500` .
- `cyclonedds_cat793f_client.xml` : `ros2 service call` no host, disca 7500 e 7501.

7.4 Armadilhas encontradas (causa confirmada)

1. **`ROS_DOMAIN_ID` herdado do perfil do shell.** O host tinha `25` e o device usava `0` ; o Cyclone descartava em silêncio ("mismatching domain"). Correção: `export ROS_DOMAIN_ID=0` explícito (`[T:14b4d291@18:57:11]`).
2. **`TCP/Port=-1` no device impede descoberta entre dois processos no device**, porque nenhum aceita conexão. Dar porta real faz o processo anunciar `10.0.2.16` (IP do guest), que o host não alcança (`[T:14b4d291@18:47:45]` , `@18:52:07`). Contorno adotado: **um único processo ROS no device**. Ponto crítico para VHAL + EVS com vários participantes (§11).
3. **Dois processos "só escuta" (sem `<Peers>`) não se descobrem.** O `sirv_cat793fsim` travava em `GetBlueprints service not available, waiting again...` (`cyclonedds_cat793f_sirvsim.xml:3-15`). Pelo menos um lado precisa discar.
4. **Um processo DDS por porta, e o device precisa discar em todas.** Os tópicos do `sirv_cat793fsim` (porta 7501) só apareceram com `cyclonedds_tcp_extra_ports` (`cat793f.yaml:23-26` , `Ros2Bridge.cpp:46-55`).
5. **Daemon do `ros2cli` com ambiente velho** esconde a topologia nova: use `ros2 daemon stop` ou `ROS2CLI_START_DAEMON=false` (`[T:14b4d291@02:14:41]` , `[T:6d19af38@14:27:16]`).
6. **Simulador iniciado sem os scripts** cai em Fast DDS/UDP e o app não o encontra. Use `launch_ue5.sh` e `launch_sirv_cat793fsim.sh` , que exportam `RMW_IMPLEMENTATION` , `ROS_DOMAIN_ID` e `CYCLONEDDS_URI` (`[T:6d19af38@14:00:43]`).

8. Desempenho medido

Só foram registradas taxas e resoluções de imagem e tamanhos de binário; latência, CPU e memória não foram medidas.

Métrica	Valor	Fonte
Taxa do simulador (fisheye JPEG,	~5.7–6.1 Hz, 480×480, 4 câmeras	`[T:14b4d291@02:21:58]` ;

`CompressedImage`)		`cat793f.yaml:17-18`
sample_bag recebido no app (`CompressedImage` JPEG)	front 1920×1080; left/right/rear 1280×720; inside/ground 640×480	`[T:14b4d291@00:58:30]`
sample_bag salvo por nó nativo	1658 frames em 6 tópicos; taxa por segundo **NÃO MEDIDA**	`scratch/received_frames/frames/`
Mensagem grande	`sensor_msgs/Image` 1920×1012 rgb8 (~5.8 MB) a cada 500 ms; recepção no device **NÃO VERIFICADA** (sem saída registrada)	`bringup_talker/fake_bag_publisher.cpp:31-55`, `image_listener.cpp:28`
String	1 Hz (talker), 2 Hz (`ros2 topic pub --rate 2`)	`talker.cpp:16`; `[T:14b4d291@18:42:13]`
Latência, CPU, memória	**NÃO MEDIDOS**	—
Tamanho do install	arm64 274 MB, x86_64 298 MB, com debug info, sem strip	`du -sh`; `file librclcpp.so`
`.so` arm64	124 arquivos, 184 MB	`find … -name '*.so'`
Binários de teste x86_64	talker 2.5 MB, listener 8.0 MB, image_listener 13 MB	`ls -la bringup_talker/build_x86_64`
APK (2 ABIs, ROS sem strip)	116 MB (14/09), 129 MB (atual)	`[T:14b4d291@00:42:12]`; `ls -la app-debug.apk`

9. Limitações e problemas conhecidos

O maior risco aberto é o alinhamento de página de 4 KB das libs ROS, que já gera aviso no AAOS 16.

1. **Alinhamento de página de 4 KB nas libs ROS.** `llvm-readelf -l install_android/rclcpp/lib/librclcpp.so` mostra `LOAD … align 0x1000`; no AAOS 16 (RPi4) aparecem `PageSizeMismatchDialog for package com.surroundvision.app` e `LOAD segment not aligned` (`[T:6d19af38@13:37:19]`). Só o `libsurroundvision_native.so` ganhou `-Wl,-z,max-page-size=16384` (`app/CMakeLists.txt:144-146`); falta recompilar o ROS com `-DCMAKE_SHARED_LINKER_FLAGS=-Wl,-z,max-page-size=16384`.
2. **Sem strip, com debug info:** os tamanhos da §8 são inflados.
3. **Caminhos absolutos do host hardcoded** em `app/CMakeLists.txt:74,76`, `build.gradle.kts:30,82,90` e nos XMLs/scripts (`launch_*.sh`). Troque antes de reproduzir.
4. **`cyclonedds_host_ip` fixo** no preset (`cat793f.yaml:32`), muda com o DHCP; o próprio comentário recomenda IP reservado (linhas 27-31).
5. **Um processo ROS por device** com a config TCP dial-out (§7.4 item 2). Dois processos no device só conversaram sem XML (UDP local, T1).
6. **`cv::cvtColor` trava** em executável nativo solto (fora de app/ART), com OpenCV Android SDK 4.10.0. **Não** se reproduz dentro do APK: logcat `CvProbe: cv::cvtColor retornou (empty=0) — SONDA CONCLUÍDA COM SUCESSO` no x86_64 e no RPi4 (`[T:6d19af38@13:59:38]`). Relevante para HAL nativa, que é justamente o caso fora de ART. Causa raiz **NÃO VERIFICADA**: OpenCL, OpenVX e threading foram descartados (`[T:14b4d291@18:23:16]`).
7. **cv_bridge** vem do branch `rolling` (§3) e compila com headers Boost avulsos (`-isystem <WS>/scratch/boost_headers_only`).
8. **Deriva do `sirv_msgs`:** 21 msgs no install contra 23 na fonte (§5).
9. **`bringup_talker/package.xml`** não declara `sensor_msgs`, embora `CMakeLists.txt:6,15-26` use; funciona com CMake puro, mas quebraria o `colcon --packages-up-to`.
10. **arm64 nunca rodou em emulador**, porque o host não suporta. O arm64 foi validado em compilação (`file` → `ARM aarch64`) e no RPi4 até a descoberta.
11. **Device real:** acesso só via adb USB; o adb por Wi-Fi não ficou disponível no build do

RPi4 (`[T:6d19af38]` , sessões de 16/09).

12. **Logging:** com `rcl_logging_noop` , o `/rosout` fica sem arquivo de log e o `RCLCPP_*` não vai para o logcat.

10. Passo a passo de reprodução (host Linux → TwistStamped → assinante no Android)

Os passos 1–8 repetem comandos que **foram executados**, com as saídas registradas. O passo 9 (TwistStamped) **NÃO FOI EXECUTADO**: não há nó TwistStamped no repositório, mas ele usa os mesmos mecanismos validados com `std_msgs/String` (E3), e `geometry_msgs` já está no build (`install_android_x86_64/geometry_msgs` , `push_x86_64/libgeometry_msgs__*.so`).

Pré-requisitos (host Ubuntu 24.04):

- colcon e CMake ≥ 3.22;
- Android SDK com `emulator` , `platform-tools` e `ndk;27.0.12077973` ;
- system image `system-images;android-34-ext9;android-automotive;x86_64` , AVD `aaos14` ;
- ROS 2 Jazzy **compilado do fonte** em `<ROS2_SRC>` , nos commits da §3: precisa existir `<ROS2_SRC>/install` nativo para as ferramentas ament/rosidl e para o `ros2` CLI.

1. Workspace e toolchain

```
mkdir -p <WS>/scratch && cd <WS>
# copiar android_x86_64.toolchain.cmake e android_arm64.toolchain.cmake deste repositório
export ANDROID_NDK_HOME=<ANDROID_SDK>/ndk/27.0.12077973
```

2. Patch do libyaml_vendor (§4.4 P1): aplicar os dois diffs em `<ROS2_SRC>/src/ros2/libyaml_vendor` .

3. Prefix path curado e cross-build do núcleo x86_64 (§4.2):

```
# gerar scratch/host_prefix_path.txt como em §4.2, depois:
export CMAKE_PREFIX_PATH="$(cat <WS>/scratch/host_prefix_path.txt)"; export
AMENT_PREFIX_PATH="$CMAKE_PREFIX_PATH"
TC="--toolchain <WS>/android_x86_64.toolchain.cmake"
colcon build --base-paths <ROS2_SRC>/src --build-base build_android_x86_64 --install-base
install_android_x86_64 \
  --packages-select libyaml_vendor --cmake-args $TC -DBUILD_TESTING=OFF -
DCMAKE_DISABLE_FIND_PACKAGE_yaml=ON
colcon build --base-paths <ROS2_SRC>/src --build-base build_android_x86_64 --install-base
install_android_x86_64 \
  --packages-up-to rclcpp geometry_msgs --packages-skip $SKIP --cmake-args $TC $CORE_ARGS --
event-handlers console_direct+
```

Esperado: o build termina sem falhas, e `file install_android_x86_64/rclcpp/lib/librclcpp.so` → `ELF 64-bit LSB shared object, x86-64` . O texto exato do resumo do colcon é **NÃO VERIFICADO** (o `logger_all.log` não traz a linha "Summary").

4. Emulador (`[T:14b4d291@17:44:48]` , `@17:45:45`). Adicione `-no-window` para rodar headless.

```
<ANDROID_SDK>/emulator/emulator -avd aaos14 -no-snapshot -no-audio -no-boot-anim -gpu
swiftshader_indirect &
adb wait-for-device
until [ "$(adb shell getprop sys.boot_completed | tr -d '\r')" = "1" ]; do sleep 3; done
adb shell getprop ro.build.version.release   # esperado: 14
adb shell getprop ro.product.cpu.abi         # esperado: x86_64
adb shell mkdir -p /data/local/tmp/ros2_android/lib
```

5. Nó assinante TwistStamped (novo, **NÃO VERIFICADO**). Crie `<WS>/bringup_talker/twist_listener.cpp` no padrão de `listener.cpp` :

```
#include "rclcpp/rclcpp.hpp"
#include "geometry_msgs/msg/twist_stamped.hpp"
class TwistListener : public rclcpp::Node {
```

```
public:
  TwistListener() : Node("android_twist_listener") {
    sub_ = create_subscription<geometry_msgs::msg::TwistStamped>("cmd_vel_stamped", 10,
      [this](const geometry_msgs::msg::TwistStamped & m) {
        RCLCPP_INFO(get_logger(), "heard twist: frame=%s lin.x=%.2f ang.z=%.2f",
          m.header.frame_id.c_str(), m.twist.linear.x, m.twist.angular.z); });
  }
private:
  rclcpp::Subscription<geometry_msgs::msg::TwistStamped>::SharedPtr sub_;
};
int main(int argc, char ** argv) {
  rclcpp::init(argc, argv); rclcpp::spin(std::make_shared<TwistListener>());
rclcpp::shutdown(); return 0;
}
```

No `bringup_talker/CMakeLists.txt`, acrescente `find_package(geometry_msgs REQUIRED)`, `add_executable(twist_listener twist_listener.cpp)`, `ament_target_dependencies(twist_listener rclcpp geometry_msgs)` e o alvo no `install(...)`. No `package.xml`, acrescente `<depend>geometry_msgs</depend>`.

6. Compilar os nós de teste (§4.2, "Nós de teste"). Saída esperada do configure: `-- Using RMW implementation 'rmw_cyclonedds_cpp'`.

7. Enviar todas as `.so`, `libc++_shared.so`, `libyaml.so` e o binário (`[T:14b4d291@17:52:04]`; a lição da §5 é mandar tudo):

```
mkdir -p <WS>/push_x86_64
find <WS>/install_android_x86_64 -name '*.so' -exec cp {} <WS>/push_x86_64/ \;
cp <NDK>/toolchains/llvm/prebuilt/linux-x86_64/sysroot/usr/lib/x86_64-linux-android/libc+
+_shared.so <WS>/push_x86_64/
adb push <WS>/push_x86_64/*.so /data/local/tmp/ros2_android/lib/
adb push <WS>/bringup_talker/build_x86_64/twist_listener /data/local/tmp/ros2_android/
adb push <WS>/cyclonedds_device.xml /data/local/tmp/ros2_android/
adb shell chmod +x /data/local/tmp/ros2_android/twist_listener
```

O `find … -exec cp` é uma simplificação, **NÃO VERIFICADA**: a sessão original usou um walk de `readelf NEEDED` mais a cópia de todas as libs (`[T:14b4d291@17:51:50]`, `@17:52:18`). Antes do passo 9, valide a cadeia com o talker, já testado (`[T:14b4d291@17:52:24]`); esperado `publishing: 'hello from android #0'`:

```
adb shell "export LD_LIBRARY_PATH=/data/local/tmp/ros2_android/lib; export
RMW_IMPLEMENTATION=rmw_cyclonedds_cpp; /data/local/tmp/ros2_android/talker"
```

8. Host: escuta TCP na 7400, domain 0 (`[T:14b4d291@18:42:13]`):

```
source <ROS2_SRC>/install/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI=file://<WS>/cyclonedds_host.xml
export ROS_DOMAIN_ID=0                # obrigatório: o perfil pode ter outro valor (§7.4 item
1)
export ROS2CLI_START_DAEMON=false
ros2 topic pub /cmd_vel_stamped geometry_msgs/msg/TwistStamped \
  "{header: {frame_id: 'base_link'}, twist: {linear: {x: 1.0}, angular: {z: 0.5}}}" --rate 2 &
ss -tlnp | grep 7400                  # esperado: LISTEN 0.0.0.0:7400 … ("ros2",…)
```

9. Android: assinante (padrão de `[T:14b4d291@18:42:13]`):

```
adb shell "export LD_LIBRARY_PATH=/data/local/tmp/ros2_android/lib; export
RMW_IMPLEMENTATION=rmw_cyclonedds_cpp; \
  export CYCLONEDDS_URI=file:///data/local/tmp/ros2_android/cyclonedds_device.xml; /data/local/
tmp/ros2_android/twist_listener"
```

Esperado por analogia, **NÃO VERIFICADO**:

```
… listener: using network interface wlan0 (tcp/10.0.2.16) selected arbitrarily from: wlan0,
eth0
[INFO] [...] [android_twist_listener]: heard twist: frame=base_link lin.x=1.00 ang.z=0.50
```

A primeira linha é real, registrada com o listener String (`[T:14b4d291@18:37:33]`): o

A primeira linha é real, registrada com o listener String ([T:14b4d291@18:38:12]); o CycloneDDS só a escreve porque o Tracing do `cyclonedds_device.xml` está ligado.

Diagnóstico se nada chegar:

- `adb shell cat /proc/net/tcp` e procure a conexão a `0202000A:1CE8` (10.0.2.2:7400 em hex), estado `01` (ESTABLISHED) (`[T:14b4d291@18:38:12]`).
- Confira o `ROS_DOMAIN_ID` dos dois lados.
- Confirme que só **um** processo ROS está rodando no device (§7.4 item 2).

Variante arm64 (device real): repita os passos 3, 6 e 7 com `android_arm64.toolchain.cmake` , `build_android` / `install_android` e `<NDK>/…/aarch64-linux-android/libc++_shared.so` , e troque `10.0.2.2` por `<HOST_IP>` no XML do device (obrigatório se o Wi-Fi filtra multicast, §7.2). Esta variante com executável nativo não foi executada no arm64; só o APK rodou no RPi4.

11. Perguntas em aberto e próximos passos (foco VHAL/EVS)

Nada desta seção foi testado; as três primeiras são os riscos centrais para embutir o ROS 2 numa HAL vendor.

1. **libc++ (CalibDroid AD-026).** Hoje tudo usa a `libc++_shared.so` do NDK, carregada do APK ou de `/data/local/tmp` . Num processo `vendor` (VHAL/EVS) há duas opções:
   - (a) empacotar a libc++_shared do NDK em `/vendor/lib64` com outro nome/namespace, lidando com o conflito com a `libc++.so` da plataforma que outras libs vendor carregam;
   - (b) recompilar o ROS 2 com Soong contra a libc++ da plataforma (`Android.bp` , `vendor: true`), portando CMake para Soong ou usando prebuilts com `sdk_version` / `vendor_available` .
   - Pergunta: qual das duas o time escolhe, e a VNDK/LL-NDK da plataforma alvo permite a (a)?
2. **SELinux.** Nunca medido. Um serviço vendor vai precisar de `.te` com `net_domain(<hal_domain>)` , permissões `tcp_socket` / `udp_socket` (`create bind connect listen accept`), possivelmente `node_bind` / `port` para 7400–7502 e leitura do XML do Cyclone em `/vendor/etc` . Sugestão: rodar primeiro em `permissive` , coletar os `avc: denied` e escrever a política a partir deles.
3. **Vários participantes DDS no mesmo device** (VHAL + EVS + app). O padrão TCP dial-out validado só funciona com **um** processo por device (§7.4 item 2). Opções a avaliar:
   - um processo "bridge" único que expõe AIDL/HIDL para VHAL/EVS;
   - Cyclone com `<TCP><Port>` real e `ExternalNetworkAddress` ;
   - UDP unicast com peers;
   - Zenoh (`rmw_zenoh`) com router.
4. **16 KB page size** (AAOS 15+/16): recompilar todo o ROS com `-Wl,-z,max-page-size=16384` (§9 item 1).
5. **Strip e tamanho:** `-DCMAKE_BUILD_TYPE=Release` (hoje vazio: `CMAKE_BUILD_TYPE:STRING=` no CMakeCache do rclcpp e do talker) e `llvm-strip` nas `.so` .
6. **Executável fora de ART:** reavaliar o hang do `cv::cvtColor` (§9 item 6) num processo vendor; uma HAL EVS é exatamente esse cenário.
7. **Init/serviço:** criar `.rc` (`service vipee_ros2_bridge /vendor/bin/… user … group inet`) e definir `CYCLONEDDS_URI` / `ROS_DOMAIN_ID` via `setenv` antes do `rclcpp::init` , como em `Ros2Bridge.cpp:31-63` . Use `shutdown_on_signal=false` se o processo tiver handlers próprios.
8. **Validação arm64 completa:** imagem chegando no RPi4 ou no head unit alvo, com medições de latência, CPU e memória (a §8 não tem).
9. **RMW:** confirmar Cyclone versus Fast DDS para o projeto. O Fast DDS não foi compilado; a escolha aqui foi pragmática (TCP via XML).
10. **Reprodutibilidade:** fixar um `.repos` com os commits da §3, versionar o workspace em git e remover os caminhos absolutos (§9 item 3).
11. **WSL em modo NAT:** não testado. Com o publicador no WSL, o device precisa discar

para `<IP do Windows>:<porta>`, com `netsh interface portproxy` ou WSL em modo `mirrored`. Hipótese a validar.